Policy Gradient Methods on CartPole: REINFORCE, Actor-Critic, and A2C

Juan Sebastián Meléndez Granados [1] **Emmanuel Oblitey Ashong** [1] **Theodoros-Marios Adalakis** [1]

Abstract

This report studies three policy-gradient reinforcement learning algorithms in the CartPole-v1 environment: REINFORCE, basic Actor-Critic, and Advantage Actor-Critic (A2C). In contrast to value-based methods from the previous assignment, these methods learn a parameterized policy directly instead of deriving actions from learned Q-values. All three algorithms were implemented using the same policy network architecture and evaluated using learning curves over environment steps. The results show that A2C reaches near-optimal performance fastest and most stably, while REINFORCE also learns a strong policy but with more variance. Basic Actor-Critic was more sensitive to hyperparameters, especially the actor learning rate, and achieved lower final stability than A2C.

1. Introduction

In the previous assignment, the CartPole-v1 environment was solved using value-based reinforcement learning methods, in particular Deep Q-Learning. These methods approximate the action-value function $Q(s, a)$ with a neural network and derive a policy indirectly by selecting actions greedily or ϵ-greedily. While effective, such approaches require additional stabilization techniques, such as target networks and experience replay.

In this assignment, we study an alternative approach: policy-gradient methods. Instead of learning action values and deriving a policy, these methods directly parameterize the policy $\pi_\theta(a|s)$ as a neural network. This allows the agent to learn stochastic policies and optimize behavior directly through gradient-based updates. However, naive policy-gradient methods can suffer from high variance, which motivates the development of Actor-Critic and advantage-based methods.

We implement and compare three algorithms: REINFORCE, basic Actor-Critic (AC), and Advantage Actor-Critic (A2C). All methods are evaluated in the CartPole-v1 environment using the same policy network architecture. The goal is to analyze how different learning signals—Monte Carlo returns, temporal-difference errors, and advantage estimates—affect learning speed, stability, and final performance.

2. Theory

2.1. Vanilla Policy Loss

The objective is defined as minimizing the expected negative action-value under the current policy (Plaat, 2022):

$$L(\theta) = \mathbb{E}_{s\sim D}\left[\mathbb{E}_{a\sim\pi_\theta(a|s)}\left[-Q^{\pi_{\theta_k}}(s,a)\right]\right]. \tag{1}$$

This objective encourages the policy to assign higher probability to actions that yield higher expected returns. However, since the expectation depends on the policy parameters θ, we cannot differentiate it directly.

To compute the gradient, we apply the likelihood-ratio trick:

$$\nabla_\theta \mathbb{E}_{a\sim\pi_\theta}[f(a)] = \mathbb{E}_{a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s)\cdot f(a)\right]. \tag{2}$$

Applying this to the loss, we obtain:

$$\nabla_\theta L(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s)\cdot(-Q(s,a))\right]. \tag{3}$$

(Sutton et al., 2000)

Rewriting:

$$\nabla_\theta L(\theta) = -\mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)\right]. \tag{4}$$

In practice, we approximate this expectation using samples and define a per-sample loss whose gradient matches this expression:

$$\mathcal{L}_{\text{policy}} = -\log \pi_\theta(a|s) Q(s,a). \tag{5}$$

This is the vanilla policy loss. It increases the probability of actions that lead to high returns and decreases the probability of actions that perform poorly.

2.2. REINFORCE

REINFORCE is the simplest policy-gradient algorithm (Williams, 1992) and directly applies the vanilla policy loss

Algorithm 1 REINFORCE

Initialize policy network π_θ
while training budget not exhausted **do**
  Run one full episode using π_θ
  Store $(s_t, a_t, r_t, \log \pi_\theta(a_t|s_t))$
  Compute discounted returns G_t
  Compute loss $\mathcal{L} = -\sum_t \log \pi_\theta(a_t|s_t) G_t$
  Update θ using gradient descent
end while

derived in the previous section.. Since the action-value function $Q^\pi(s, a)$ is unknown, it is estimated using Monte Carlo returns obtained from complete episodes.

For each timestep t, the return is defined as:

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k, \tag{6}$$

which represents the total discounted reward obtained from timestep t until the end of the episode.

By substituting this estimate into the policy loss, we obtain:

$$\mathcal{L}_{\text{REINFORCE}} = -\sum_t \log \pi_\theta(a_t|s_t) G_t. \tag{7}$$

The key characteristic of REINFORCE is that updates are performed only after a full episode has been observed. This is necessary because the return G_t depends on future rewards, which are not available until the episode terminates.

REINFORCE provides an unbiased estimate of the policy gradient. However, because it relies on full-episode returns, it typically exhibits high variance and slower convergence compared to methods that use additional value estimates.

2.3. Actor-Critic

The REINFORCE algorithm estimates the action-value $Q^\pi(s, a)$ using full-episode returns, which leads to high variance. Actor-Critic methods address this by introducing a second model, called the *critic*, that estimates the value of a state (Sutton & Barto, 2018).

The actor is the policy network $\pi_\theta(a|s)$, which selects actions, while the critic is a value network $V_\phi(s)$ that estimates the expected return from a state s.

Instead of using Monte Carlo returns, Actor-Critic uses a one-step approximation of the action-value:

$$Q(s_t, a_t) \approx r_t + \gamma V_\phi(s_{t+1}), \tag{8}$$

where s_t is the current state, a_t the action taken, r_t the reward obtained, and s_{t+1} the next state.

Algorithm 2 Basic Actor-Critic

Initialize actor π_θ and critic V_ϕ
Observe initial state s
while training budget not exhausted **do**
  Sample action $a \sim \pi_\theta(\cdot|s)$
  Execute a, observe reward r and next state s'
  Compute TD target $y = r + \gamma V_\phi(s')$
  Compute TD error $\delta = y - V_\phi(s)$
  Update actor using $-\log \pi_\theta(a|s)\delta$
  Update critic using $(V_\phi(s) - y)^2$
  Set $s \leftarrow s'$ (reset if terminal)
end while

This leads to the temporal-difference (TD) error:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \tag{9}$$

which measures how much better or worse the observed outcome was compared to the critic's estimate.

The TD error is used as a learning signal for both networks. The actor is updated using:

$$\mathcal{L}_{\text{actor}} = -\log \pi_\theta(a_t|s_t)\delta_t, \tag{10}$$

while the critic is trained to minimize the squared error:

$$\mathcal{L}_{\text{critic}} = \left(V_\phi(s_t) - (r_t + \gamma V_\phi(s_{t+1}))\right)^2. \tag{11}$$

Unlike REINFORCE, updates are performed after every step, since the TD target depends only on the immediate reward and the next state.

A key difference with value-based methods is that we cannot use the Q-learning target $r + \gamma \max_{a'} Q(s', a')$, since this assumes a greedy policy. Actor-Critic instead evaluates the current stochastic policy, making the TD target consistent with the behavior of the agent.

Actor-Critic reduces the variance of REINFORCE by using a learned value estimate. However, because the actor relies on the critic's predictions, the method can be sensitive to the quality of the value function and to the choice of learning rates.

2.4. Advantage Actor-Critic (A2C)

In all policy-gradient methods considered so far, the policy is updated using an estimate of the action-value $Q^\pi(s, a)$. The difference between the methods lies in how this quantity is estimated. REINFORCE uses Monte Carlo returns, while Actor-Critic uses a bootstrapped estimate based on the value function.

A key improvement is to subtract a baseline from this estimate. Instead of using $Q^\pi(s, a)$ directly, we consider how

much better an action is compared to the expected outcome at that state. This leads to the *advantage function*, defined as:

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s), \tag{12}$$

where $V^\pi(s)$ denotes the expected return from state s.

The advantage measures how much better or worse an action a is compared to the average outcome at state s. If $A(s,a)$ is positive, the action performed better than expected; if it is negative, it performed worse. Using advantages instead of raw Q-values reduces variance because the baseline $V^\pi(s)$ removes state-dependent effects that are unrelated to the action choice. Importantly, subtracting this baseline does not introduce bias, since it does not depend on the action.

In Advantage Actor-Critic (A2C), we combine the idea of a learned value function with Monte Carlo returns (Sutton & Barto, 2018). As in REINFORCE, the action-value is estimated using the return:

$$Q^\pi(s_t, a_t) \approx G_t, \tag{13}$$

where G_t is the discounted sum of future rewards starting from timestep t.

The advantage is then computed as:

$$A_t = G_t - V_\phi(s_t), \tag{14}$$

where $V_\phi(s_t)$ is the critic's estimate of the value of state s_t.

The actor is updated using:

$$\mathcal{L}_{\text{actor}} = -\sum_t \log \pi_\theta(a_t|s_t) A_t, \tag{15}$$

and the critic is trained to regress towards the return:

$$\mathcal{L}_{\text{critic}} = \sum_t \left(V_\phi(s_t) - G_t\right)^2. \tag{16}$$

Like REINFORCE, A2C performs updates after a full episode, since the return G_t requires future rewards. However, the use of the advantage significantly stabilizes learning by reducing the variance of the policy-gradient estimate.

A2C can be interpreted as a variance-reduced version of REINFORCE, where the critic acts as a learned baseline. This results in faster and more stable convergence in practice.

Algorithm 3 Advantage Actor-Critic (A2C)

Initialize actor π_θ and critic V_ϕ
while training budget not exhausted **do**
  Run one full episode using π_θ
  Store $(s_t, a_t, r_t, \log \pi_\theta(a_t|s_t))$
  Compute discounted returns G_t
  Compute values $V_\phi(s_t)$
  Compute advantages $A_t = G_t - V_\phi(s_t)$
  Update actor using $-\sum_t \log \pi_\theta(a_t|s_t) A_t$
  Update critic using $\sum_t (V_\phi(s_t) - G_t)^2$
end while

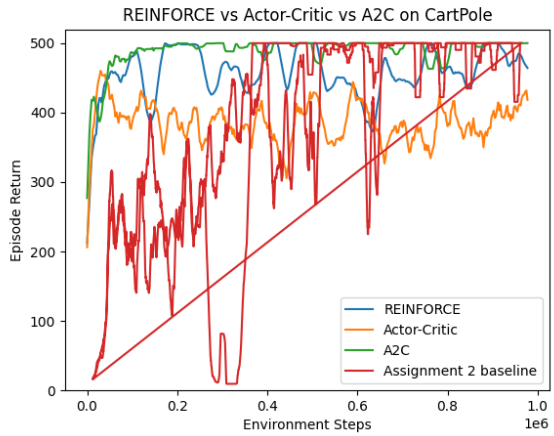

Figure 1. Learning curves for REINFORCE, Actor-Critic, and A2C on CartPole-v1. Curves show average evaluation return over environment steps. A2C reaches near-optimal performance fastest and remains the most stable.

3. Experiments

3.1. Experimental Setup

All experiments were performed in the CartPole-v1 environment. The agent observes a four-dimensional continuous state describing the cart and pole dynamics, and selects one of two discrete actions: pushing the cart left or right. The reward is $+1$ for every timestep the pole remains balanced, and the maximum episode return is 500.

All three algorithms used the same policy network architecture: a multilayer perceptron with two hidden layers, ReLU activations, and a softmax output over the two actions. Actor-Critic and A2C additionally used a value network with the same hidden-layer structure and a scalar output. Each algorithm was trained for 10^6 environment steps. Evaluation was performed every 2500 environment steps. Each configuration was repeated five times, and the plotted curve shows the average smoothed return.

For Actor-Critic, an actor learning rate of 10^{-4} and critic learning rate of 10^{-3} were used. This was chosen because preliminary experiments showed that using the same learning rate for both networks could destabilize the actor and lead to policy collapse.

3.2. Results

The results in Figure 1 show that A2C obtains the strongest performance. It rapidly reaches returns close to the maximum score of 500 and remains stable for most of training. REINFORCE also learns a high-performing policy, but it converges more slowly and shows more oscillation. This is consistent with the high variance expected from Monte Carlo policy-gradient estimates.

Basic Actor-Critic improves quickly at the beginning of training, but it does not reach the same final stability as A2C. Its learning curve remains below the other two policy-gradient methods for much of the run. This suggests that the one-step TD learning signal reduces the delay of REINFORCE but introduces sensitivity to the quality of the critic and the actor learning rate.

The value-based results from the previous assignment are included in the appendix as a reference. Even the best DQN configuration, using both target networks and experience replay, showed more irregular behavior than the best policy-gradient method in this assignment. This comparison suggests that directly optimizing the policy can be very effective in CartPole, especially when combined with a variance-reducing baseline.

4. Discussion

The experiments show a clear advantage for A2C. Its improvement over REINFORCE can be explained by the use of a learned value baseline. REINFORCE updates the policy using raw Monte Carlo returns, so all variation in episode outcomes directly affects the policy gradient. A2C instead uses the advantage $G_t - V(s_t)$, which centers the learning signal around the expected return from each state. This reduces variance and produces more stable learning.

The basic Actor-Critic implementation showed that bootstrapping can be useful but also fragile. Since AC updates after every step, noisy TD errors can immediately affect the actor. During development, we observed that using an actor learning rate that was too high caused the policy to collapse after initially improving. Reducing the actor learning rate stabilized training, but AC still remained less reliable than A2C. This illustrates an important practical point: Actor-Critic methods depend strongly on the interaction between actor and critic updates.

Compared to the previous value-based assignment, the policy-gradient methods required a different form of stabilization. DQN relied on experience replay and target networks to stabilize Q-learning. In contrast, A2C used a baseline to reduce variance in the policy-gradient estimate. Both approaches address instability, but they do so in different ways: DQN stabilizes value targets, while A2C stabilizes the policy update signal.

A limitation of this study is that only one environment was tested. CartPole is relatively simple, with a low-dimensional state space and only two actions. Therefore, the conclusions may not transfer directly to environments with larger action spaces, sparse rewards, or more complex dynamics. In addition, only a limited set of hyperparameters was tested. More systematic tuning could further improve the Actor-Critic results.

5. Conclusion

This assignment implemented and compared REINFORCE, basic Actor-Critic, and A2C in CartPole-v1 using a shared policy network. The results show that A2C achieved the best overall performance, reaching near-optimal returns faster and more stably than the other methods. REINFORCE was simple and effective but suffered from higher variance, while Actor-Critic was more sensitive to hyperparameters and less stable.

Overall, the experiments demonstrate the importance of the learning signal used to update the policy. Monte Carlo returns provide an unbiased but noisy signal, TD errors allow online updates but can be unstable, and advantages provide a more stable centered signal. Future work could extend this comparison by testing entropy regularization, multi-step returns, parallel environments, or additional control tasks beyond CartPole.

References

Plaat, A. *Deep reinforcement learning*, volume 10. Springer, 2022.

Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT press, 2018.

Sutton, R. S., McAllester, D. A., Singh, S. P., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 2000.

Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.

A. Previous Assignment Reference

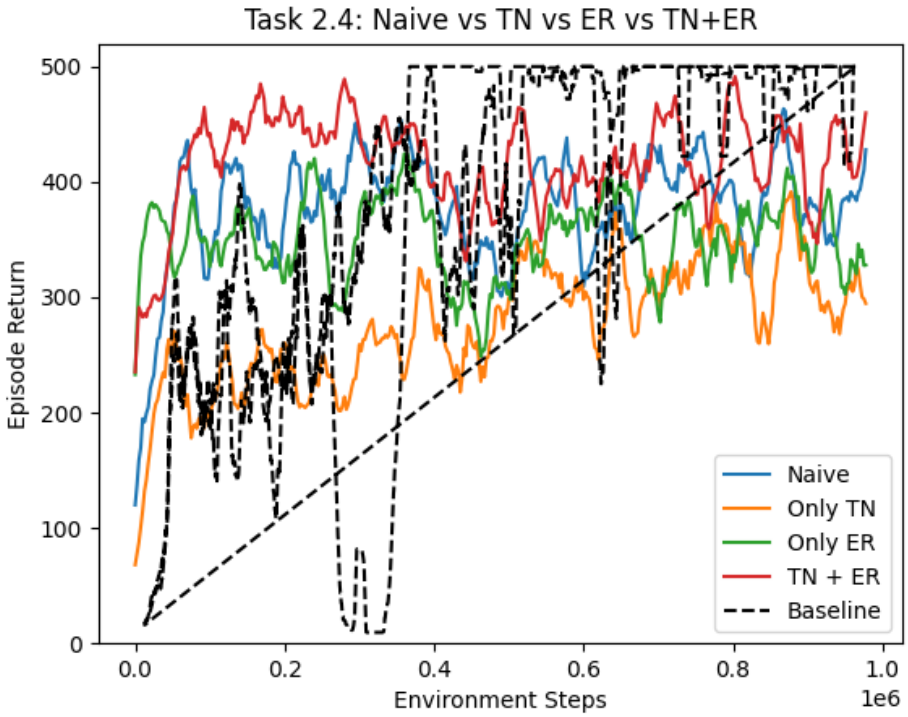

Figure 2. Learning curves from the previous assignment comparing DQN configurations: Naive DQN, target network only, experience replay only, and target network with experience replay. The TN+ER configuration achieved the strongest value-based performance and is used as qualitative reference for this assignment.